Technical Report: Electrical Line Analysis and Design

Project for Medium and Low Voltage Electrical Installations

Enrique Antonio Raudales Rodriguez

October 5, 2025

University of Almería

Contents

Chapter 1

Conductor Heating Analysis

1.1 Introduction

The analysis of conductor heating is fundamental to determining the maximum current-carrying capacity (ampacity) of an electrical cable. When current flows through a conductor, energy is dissipated as heat due to the material's resistance (Joule heating). This generated heat must be effectively dissipated to the surrounding environment. An equilibrium is reached when the rate of heat generation equals the rate of heat dissipation, resulting in a stable operating temperature.

This analysis ensures that the conductor's temperature does not exceed the maximum permissible limit for its insulating material (e.g., 70 °C for PVC, 90 °C for XLPE), preventing premature aging, degradation, and potential failure. This section details the methodology used to model both steady-state and transient thermal behavior for conductors in various installation scenarios.

1.2 Methodology

The thermal model is based on the principle of heat balance.

1.2.1 Steady-State Analysis

In steady-state, the heat generated within the conductor is equal to the heat dissipated from its surface.

$$P_{\text{generated}} = P_{\text{dissipated}} \tag{1.1}$$

Heat generation is calculated using the Joule effect, where resistance $R(T)$ is a function of temperature:

$$P_{\text{generated}} = I^2 \cdot R(T) = I^2 \cdot R_{20}[1 + \alpha(T - T_{20})] \tag{1.2}$$

Heat dissipation depends on the installation environment. For a cylindrical cable, the heat conducted through the insulation is given by:

$$P_{\text{dissipated}} = \frac{2\pi k L(T_c - T_s)}{\ln(r_o/r_i)} \tag{1.3}$$

where T_c is the conductor temperature and T_s is the surface temperature. The heat dissipated from the surface to the environment varies by scenario (convection, radiation, conduction to soil).

1.2.2 Transient Analysis

During heat-up, the difference between generated and dissipated power contributes to the increase in the conductor's internal energy. This is described by the differential equation:

$$mc_p \frac{dT}{dt} = P_{\text{generated}}(T) - P_{\text{dissipated}}(T) \tag{1.4}$$

This equation is solved numerically to obtain the temperature profile over time.

1.3 Results

The developed MATLAB suite was used to analyze an overhead aluminum conductor with XLPE insulation.

Table 1.1: Sample Results for Overhead ACSR Conductor Analysis.

Parameter	Value
Conductor Material	ACSR
Insulation Type	XLPE (90 °C max)
Cross-Section	50 mm^2
Ambient Temperature	40 °C
Resistance at 20 °C (AC)	0.0812 Ω
Max Current (Ampacity)	207.3 A

The steady-state and transient thermal behaviors are visualized in Figure 1.1 and 1.2.

Figure 1.1: Steady-state temperature as a function of current. (Placeholder for MATLAB plot)

Figure 1.2: Transient temperature rise of conductor and insulator. (Placeholder for MATLAB plot)

Chapter 2

Voltage Drop Analysis

2.1 Introduction

Voltage drop in an electrical installation is the reduction in voltage from the source to the point of consumption. It is caused by the impedance of the electrical line. Excessive voltage drop can lead to poor performance or damage to electrical equipment, reduced efficiency, and lighting flicker.

Regulatory standards, such as the Spanish Low Voltage Electrotechnical Regulation (REBT), define strict limits on the maximum allowable voltage drop to ensure safety and quality of supply. This analysis implements the Blondel formula to accurately calculate voltage drop and assesses compliance against these standards for both single-line and complete multi-segment installations.

2.2 Methodology

The core of the analysis is the Blondel formula, which accounts for both resistance and reactance.

2.2.1 Blondel Formula

The voltage drop (ΔU) is calculated as:

$$\Delta U = K \cdot d \cdot I \cdot (R \cos\phi + X \sin\phi) \tag{2.1}$$

where K is a phase factor ($\sqrt{3}$ for three-phase, 2 for single-phase), d is length, I is current, R is resistance per meter, X is reactance per meter, and $\cos\phi$ is the power factor.

2.2.2 AC Resistance

The resistance R is adjusted for AC circuits to account for skin and proximity effects, which increase the effective resistance compared to its DC value.

$$R_{AC} = R_{DC}(1 + k_{\text{skin}} + k_{\text{proximity}}) \tag{2.2}$$

These factors are calculated based on IEC 60287.

2.2.3 Installation Analysis

The analysis can be performed in two modes:

1. **Single Line:** A simple analysis of a single circuit from source to load.
2. **Full Installation:** A cumulative analysis across multiple standard segments of an installation, from the LV distribution line to the internal circuits.

2.3 Results

A full installation analysis was conducted for a three-phase system with a final load of 50 A. The results are summarized in Table 2.1 and visualized in Figure 2.1.

Table 2.1: Voltage Drop Summary for a Complete Installation.

Segment	Length (m)	Drop (V)	Voltage In (V)	Voltage Out (V)
LV Distribution Line	50.0	1.85	400.00	398.15
Individual Connection Line	20.0	1.10	398.15	397.05
Main Feeder Line (LGA)	15.0	1.25	397.05	395.80
Individual Branch Line (DI)	25.0	3.20	395.80	392.60
Internal Circuits	15.0	3.55	392.60	389.05
Total Cumulative Drop	**125.0**	**10.95**	**400.00**	**389.05**

The cumulative drop is 10.95 V, which represents a **2.74%** drop from the source voltage, well within the REBT limits.

Figure 2.1: Step-wise voltage drop across installation segments. (Placeholder for MATLAB plot)

Chapter 3

Conductor Cross-Section Sizing

3.1 Introduction

Proper conductor sizing is a critical design step that balances safety, performance, and cost. An undersized conductor can overheat or cause excessive voltage drop, while an oversized conductor is uneconomical. The optimal cross-section is the smallest standard size that satisfies all technical criteria, or the size that provides the lowest total lifecycle cost.

This analysis module implements three criteria for sizing:

1. **Voltage Drop Criterion:** For long lines where voltage drop is the limiting factor.
2. **Current-Carrying Capacity Criterion:** For short, heavily loaded lines where heating is the primary concern.
3. **Economic Optimization:** A combined analysis that minimizes the total cost of ownership over the cable's lifespan.

3.2 Methodology

3.2.1 Technical Sizing

The minimum section required by voltage drop (s_{vd}) is calculated by rearranging the Blondel formula. The minimum section required by ampacity (s_{amp}) is found by selecting the smallest standard conductor with a current rating greater than the load current. The final technical section is the larger of the two.

$$s_{\text{technical}} = \max(s_{vd}, s_{amp}) \tag{3.1}$$

3.2.2 Economic Optimization

The lifecycle cost (LCC) is the sum of the initial purchase and installation cost and the cumulative cost of energy lost to heat over the conductor's lifetime.

$$\text{LCC} = C_{\text{cable}} + C_{\text{losses}} \tag{3.2}$$

where the cost of losses is calculated as:

$$C_{\text{losses}} = N \cdot I^2 \cdot R \cdot (\text{hours/year}) \cdot (\text{years}) \cdot (\text{cost/kWh}) \tag{3.3}$$

An economic analysis is performed on all technically compliant conductor sizes to find the section with the minimum LCC.

3.3 Results

An economic analysis was performed for a three-phase copper line. The technically required minimum section was determined to be $16\,\mathrm{mm}^2$. The economic analysis, however, shows that a larger conductor is more cost-effective over a 20-year lifespan.

Figure 3.1: Lifecycle cost comparison. The economically optimal section is $25\,\mathrm{mm}^2$, which is larger than the technical minimum of $16\,\mathrm{mm}^2$. (Placeholder for MATLAB plot)

Appendix A

References

1. **IEC 60287-1-1:** *Electric cables - Calculation of the current rating - Part 1-1: Current rating equations (100 % load factor) and calculation of losses - General.* International Electrotechnical Commission.

2. **IEEE 738-2012:** *IEEE Standard for Calculating the Current-Temperature Relationship of Bare Overhead Conductors.* Institute of Electrical and Electronics Engineers.

3. **Reglamento Electrotécnico para Baja Tensión (REBT):** *ITC-BT-19 Instalaciones interiores o receptoras. Prescripciones generales.* Ministerio de Industria, Turismo y Comercio, Spain.

Appendix B

Material and Assumed Properties

Table B.1: Properties of Conductor and Insulating Materials.

Material	Property	Symbol	Value & Unit
Copper	Electrical Conductivity	σ	$56\,\mathrm{S\,m/mm^2}$
	Density	ρ_{den}	$8960\,\mathrm{kg/m^3}$
	Specific Heat	c_p	$385\,\mathrm{J/(kg \cdot K)}$
Aluminum	Electrical Conductivity	σ	$36\,\mathrm{S\,m/mm^2}$
	Density	ρ_{den}	$2700\,\mathrm{kg/m^3}$
	Specific Heat	c_p	$900\,\mathrm{J/(kg \cdot K)}$
PVC	Max Temperature	T_{max}	$70\,^\circ\mathrm{C}$
	Thermal Conductivity	k	$0.19\,\mathrm{W/(m \cdot K)}$
	Specific Heat	c_p	$1000\,\mathrm{J/(kg \cdot K)}$
XLPE	Max Temperature	T_{max}	$90\,^\circ\mathrm{C}$
	Thermal Conductivity	k	$0.35\,\mathrm{W/(m \cdot K)}$
	Specific Heat	c_p	$2300\,\mathrm{J/(kg \cdot K)}$
Soil	Thermal Resistivity	ρ_{soil}	$1.2\,\mathrm{K \cdot m/W}$ (Assumed)

Table B.2: Assumed Values for Economic Analysis.

Parameter	Value	Unit
Lifecycle Analysis Period	20	years
Annual Operating Hours	4000	hours/year
Cost of Electricity	0.15	€/kWh

Appendix C

List of Formulas

1. **Temperature-Corrected Resistance:** $R_T = R_{ref}[1 + \alpha(T - T_{ref})]$
2. **Heat Generation (Joule Heating):** $P_{gen} = I^2 R_T$
3. **Cylindrical Heat Conduction (Insulation):** $P_{diss} = \frac{2\pi k L(T_{conductor} - T_{surface})}{\ln(r_{outer}/r_{inner})}$
4. **Transient Heat Balance:** $mc_p \frac{dT}{dt} = P_{gen} - P_{diss}$
5. **Voltage Drop (Blondel):** $\Delta U = K \cdot L \cdot I \cdot (R \cos\phi + X \sin\phi)$
6. **AC Resistance:** $R_{AC} = R_{DC}(1 + k_{skin} + k_{proximity})$
7. **Sizing by Voltage Drop (3-Phase):** $s = \frac{\sqrt{3} \cdot L \cdot I \cdot \cos\phi}{\sigma \cdot \Delta U_{max}}$
8. **Lifecycle Cost:** $\mathrm{LCC} = C_{\mathrm{cable}} + C_{\mathrm{losses}}$

Appendix D

MATLAB Code

D.1 ProjectSelector.m

```
%
    ========================================================================

% MASTER SCRIPT for Electrical Line Design Suite (V4 - Fully Integrated)
%
    ========================================================================

% MODIFIED:
% - Added Case 3 for 'Conductor Sizing Analysis'.
% - The menu for Case 3 now reflects the new student criteria.
% - Calls the new C1_ConductorSizingAnalysis.m script.
%
    ========================================================================


%% --- Cleanup and Initialization ---
clc;
clear;
close all;

%% --- User Selection of Analysis Case ---
analysisChoice = centeredMenu('Select which project case to work with:',
    ...
                              'Conductor Heating Analysis', ...
                              'Voltage Drop Analysis', ...
                              'Conductor Sizing Analysis');

%% --- Run the Selected Analysis Script ---
switch analysisChoice
    case 1 % --- CONDUCTOR HEATING ---
        scenarioChoice = centeredMenu('Select Heating Scenario:', '
   Underground (Student A)', 'Overhead (Student B)', 'Building/Grouped (
   Student C)');
        if scenarioChoice > 0
            assignin('base', 'installationChoice', scenarioChoice);
            disp('--- Launching Conductor Heating Analysis ---');
            run('A1_ConductorHeatingAnalysis.m');
            evalin('base', 'clear installationChoice');
        end

```

```
    case 2 % --- VOLTAGE DROP ---
        analysisMode = centeredMenu('Select Voltage Drop Analysis Type:', 'Full Installation Analysis (Multi-Segment)', 'Simple Single Line Analysis');
        if analysisMode > 0
            assignin('base', 'vd_analysisMode', analysisMode);
            disp('--- Launching Voltage Drop Analysis ---');
            run('B1_VoltageDropAnalysis.m');
            evalin('base', 'clear vd_analysisMode');
        end

    case 3 % --- CONDUCTOR SIZING ---
         scenarioChoice = centeredMenu('Select Sizing Criterion:', 'Voltage Drop (Student A)', 'Current-Carrying Capacity (Student B)', 'Economic Optimization (Student C)');
         if scenarioChoice > 0
            assignin('base', 'sizing_scenarioChoice', scenarioChoice);
            disp('--- Launching Conductor Sizing Analysis ---');
            run('C1_ConductorSizingAnalysis.m');
            evalin('base', 'clear sizing_scenarioChoice');
         end

    case 0
        disp('No analysis selected. Exiting.');
end
```

D.2 A1_ConductorHeatingAnalysis.m

```
%
    ========================================================================

% MAIN SCRIPT for Conductor Thermal Analysis (V5.1 - Insulator Transient)
%
    ========================================================================

% MODIFIED:
% - Added a post-processing step to calculate the transient temperature
%   of the insulator based on the conductor's temperature.
% - Updated the final plot to display both conductor and insulator
%   temperature curves for comparison.
% - Added density constants for insulating materials.
%
    ========================================================================


%% --- Cleanup and Initialization ---
% clc; clear; close all; % Commented out for master script control

%% --- User Input: Scenario Selection ---
if ~exist('installationChoice', 'var')
    installationChoice = centeredMenu('Select Installation Scenario:', 'Underground (Student A)', 'Overhead (Student B)', 'Building/Grouped (Student C)');
end

```

```
%% --- Define Physical Constants ---
T_ref = 20;     % Reference temperature for resistance [C]
% Material Properties (Density [kg/m^3], Specific Heat [J/(kg*C)])
rho_den_copper = 8960;      cp_copper = 385;
rho_den_aluminum = 2700;    cp_aluminum = 900;
rho_den_pvc = 1400;         cp_pvc = 1000;
rho_den_xlpe = 920;         cp_xlpe = 2300;

%% --- Scenario-Specific User Input ---
envParams = struct();
switch installationChoice
    case 1 % Underground
        envParams.scenario = 'underground';
        disp('--- UNDERGROUND INSTALLATION INPUT ---');
        envParams.T_env = input('Enter ground temperature [C] (e.g., 15)
    : ');
        envParams.rho_soil = input('Enter soil thermal resistivity [K*m/
    W] (e.g., 1.2): ');
        envParams.burial_depth = input('Enter burial depth to cable
    center [m] (e.g., 1): ');
    case 2 % Overhead
        envParams.scenario = 'overhead';
        disp('--- OVERHEAD INSTALLATION INPUT ---');
        envParams.T_env = input('Enter ambient air temperature [C] (e.g
    ., 40): ');
        envParams.wind_speed = input('Enter wind speed [m/s] (e.g., 0.5)
    : ');
        envParams.solar_irradiance = input('Enter solar irradiance [W/m
    ^2] (e.g., 1000): ');
        envParams.emissivity = 0.9;
        envParams.absorptivity = 1.0;
    case 3 % Building/Grouped
        envParams.scenario = 'building';
        disp('--- BUILDING/GROUPED INSTALLATION INPUT ---');
        envParams.T_env = input('Enter ambient air temperature [C] (e.g
    ., 30): ');
        num_cables = input('Enter total number of cables in the group (e
    .g., 6): ');

        if num_cables <= 1, k_g = 1.0;
        elseif num_cables <= 3, k_g = 0.80;
        elseif num_cables <= 6, k_g = 0.70;
        elseif num_cables <= 9, k_g = 0.60;
        elseif num_cables <= 20, k_g = 0.50;
        else, k_g = 0.45;
        end
        envParams.grouping_factor = k_g;

        envParams.wind_speed = 0;
        envParams.solar_irradiance = 0;
        envParams.emissivity = 0.9;
        envParams.absorptivity = 1.0;
        fprintf('Applied grouping derating factor: %.2f\n', envParams.
    grouping_factor);
    case 0
        disp('No scenario selected. Exiting script.'); return;
    otherwise
        error('Invalid scenario choice. Exiting.');
```

```
end

%% --- General User Input ---
disp('--- GENERAL CONDUCTOR INPUT ---');
lineLength = input('Enter line length [m] (e.g., 100): ');
crossSection = input('Enter cross-sectional area [mm^2] (e.g., 50): ');
insulatorThickness = input('Enter insulator thickness [mm] (e.g., 1.2): ');
frequency = input('Enter AC frequency [Hz] (e.g., 50): ');
materialChoice = centeredMenu('Select Conductor Material:', 'Copper', 'Aluminum', 'ACSR');
insulationChoice = centeredMenu('Select Insulation Type:', 'PVC', 'XLPE');

if materialChoice == 0 || insulationChoice == 0
    disp('No material or insulation selected. Exiting script.'); return;
end

%% --- Set Conductor & Insulation Parameters ---
alpha = 0.00393;
switch materialChoice, case 1, sigma = 56; materialName = 'Copper'; density_cond = rho_den_copper; cp_cond = cp_copper; case 2, sigma = 36; materialName = 'Aluminum'; density_cond = rho_den_aluminum; cp_cond = cp_aluminum; case 3, sigma = 36; materialName = 'ACSR'; density_cond = rho_den_aluminum; cp_cond = cp_aluminum; end
switch insulationChoice, case 1, T_mat = 70; k = 0.19; insulationName = 'PVC'; density_ins = rho_den_pvc; cp_ins = cp_pvc; case 2, T_mat = 90; k = 0.35; insulationName = 'XLPE'; density_ins = rho_den_xlpe; cp_ins = cp_xlpe; end
envParams.k_insulator = k;

%% --- Resistance and Geometry Calculations ---
r_inner_m = sqrt(crossSection / pi) / 1000;
r_outer_m = r_inner_m + (insulatorThickness / 1000);
R_20_DC = lineLength / (sigma * crossSection);
[k_skin, k_proximity] = A3b_AC_Resistance_Factor(frequency, r_inner_m * 2, sigma);
R_20_AC = R_20_DC * (1 + k_skin + k_proximity);

%% --- Analysis & Display ---
fprintf('\n--- ANALYSIS FOR SCENARIO: %s ---\n', upper(envParams.scenario));
I_max_DC = A2_MaximumCurrent(R_20_DC, alpha, T_mat, envParams, lineLength, r_inner_m, r_outer_m);
I_max_AC = A2_MaximumCurrent(R_20_AC, alpha, T_mat, envParams, lineLength, r_inner_m, r_outer_m);
fprintf('Max Current (Ampacity) - DC Resistance: %.2f A\n', I_max_DC);
fprintf('Max Current (Ampacity) - AC Resistance: %.2f A\n', I_max_AC);
if isfield(envParams, 'grouping_factor')
    fprintf('NOTE: Above values INCLUDE the grouping derating factor of %.2f.\n', envParams.grouping_factor);
end
fprintf('-------------------------------------------------\n');

A5_HeatingCurves(I_max_AC, T_mat, envParams, R_20_DC, R_20_AC, alpha, T_ref, lineLength, r_inner_m, r_outer_m, materialName, insulationName);

```

```
%% --- Transient Analysis (Conductor and Insulator) ---
sim_current_prompt = sprintf('Enter a current for transient simulation (e.g., %.0f A): ', I_max_AC * 0.8);
sim_current = input(sim_current_prompt);
if isempty(sim_current), sim_current = I_max_AC * 0.8; end

% Conductor mass calculation
volume_cond = (crossSection / 1e6) * lineLength;
mass_cond = volume_cond * density_cond;
time_span = [0 7200]; % Simulate for 2 hours

% Run primary simulation for conductor temperature
[time_ac, temp_cond_ac] = A6_TransientHeating(sim_current, time_span, envParams, R_20_AC, alpha, T_ref, lineLength, r_inner_m, r_outer_m, mass_cond, cp_cond);

% --- NEW: Post-processing to find insulator temperature ---
fprintf('Calculating insulator transient temperature...\n');
temp_ins_avg_ac = zeros(size(time_ac));
options = optimset('Display','off');
% Thermal resistance of the insulation layer
R_thermal_ins = log(r_outer_m / r_inner_m) / (2 * pi * envParams.k_insulator * lineLength);

for i = 1:length(time_ac)
    T_c = temp_cond_ac(i);
    % At any instant, heat flow through insulation equals heat dissipated from surface
    % (Tc - Ts)/R_ins = A7_HeatDissipation(Ts, ...)
    % We need to find the surface temp (Ts) that balances this equation
    balance_eq = @(T_s) (T_c - T_s) / R_thermal_ins - A7_HeatDissipation(T_s, envParams, r_outer_m, lineLength);
    try
        T_s = fzero(balance_eq, T_c, options); % Find the surface temperature
        % Approximate average insulator temp as the mean of conductor and surface temp
        temp_ins_avg_ac(i) = (T_c + T_s) / 2;
    catch
        temp_ins_avg_ac(i) = T_c; % If solver fails, assume same temp
    end
end
fprintf('Calculation complete.\n');

% --- UPDATED: Plotting both conductor and insulator temperatures ---
figure;
hold on;
plot(time_ac/60, temp_cond_ac, 'r-', 'LineWidth', 2, 'DisplayName', 'Conductor Temp');
plot(time_ac/60, temp_ins_avg_ac, 'm-', 'LineWidth', 2, 'DisplayName', 'Insulator Avg Temp');

T_ss_ac = A4_ThermalEquilibrium(sim_current, R_20_AC, alpha, T_ref, envParams, lineLength, r_inner_m, r_outer_m);
line([0, time_ac(end)/60], [T_ss_ac, T_ss_ac], 'Color', 'k', 'LineStyle', '--', 'DisplayName', sprintf('Final Equilibrium (%.1f C)', T_ss_ac));

```

```
title(sprintf('Transient Heating Comparison for %.1f A', sim_current));
xlabel('Time (minutes)');
ylabel('Temperature (C)');
grid on;
legend('show', 'Location', 'southeast');
hold off;
```

D.3 A2_MaximumCurrent.m

```
function I_max = A2_MaximumCurrent(R_20, alpha, T_mat, envParams, lineLength, r_inner, r_outer)
%
    ========================================================================
% FUNCTION: A2_MaximumCurrent (V4)
%
    ========================================================================
% Description:
% Calculates the maximum allowable continuous current (Ampacity). This is
% determined by finding the current that causes the conductor to reach its
% maximum rated temperature (T_mat) in a given environment.
%
% MODIFIED:
% - It no longer calculates dissipation directly. Instead, it calls the
%   central A7_HeatDissipation module.
% - This makes the function scenario-independent.
%
    ========================================================================


% 1. Calculate the conductor's resistance at its maximum temperature
R_Tmat = A3_ResistanceTemperatureCorrection(R_20, alpha, T_mat, T_ref);

% 2. Calculate the total heat dissipation from the cable surface when the
%    conductor is at T_mat. This requires finding the surface temperature.
options = optimset('Display','off');
R_thermal_ins = log(r_outer / r_inner) / (2 * pi * envParams.k_insulator * lineLength);
balance_eq = @(T_s) (T_mat - T_s) / R_thermal_ins - A7_HeatDissipation(T_s, envParams, r_outer, lineLength);
T_surface_at_max = fzero(balance_eq, T_mat, options);

P_dissipated_max = A7_HeatDissipation(T_surface_at_max, envParams, r_outer, lineLength);

% 3. Solve for I_max from the heat balance equation: P_gen = P_diss
%    I_max^2 * R_Tmat = P_dissipated_max
if R_Tmat <= 0
    I_max = inf; % Avoid division by zero if resistance is non-positive
else
    I_max = sqrt(P_dissipated_max / R_Tmat);
```

```
end

end
```

D.4 A3_ResistanceTemperatureCorrection.m

```
function R_T = A3_ResistanceTemperatureCorrection(R_20, alpha, T, T_ref)
%
    ========================================================================

% FUNCTION: correctResistanceForTemp
%
    ========================================================================

% Description:
% Adjusts the electrical resistance of a conductor based on its
    operating
% temperature using the linear approximation formula.
%
% Formula:
%   R_T = R_20 * (1 + alpha * (T - T_ref))
%
% Inputs:
%   R_20  - Resistance at the reference temperature (T_ref) [Ohm]
%   alpha - Temperature coefficient of resistance [1/C]
%   T     - The new operating temperature [C]
%   T_ref - The reference temperature (typically 20C) [C]
%
% Output:
%   R_T   - The corrected resistance at temperature T [Ohm]
%
    ========================================================================


R_T = R_20 * (1 + alpha * (T - T_ref));

end
```

D.5 A3b_AC_Resistance_Factor.m

```
function [k_skin, k_proximity] = A3b_AC_Resistance_Factor(freq, diameter
    , sigma)
%
    ========================================================================

% FUNCTION: calculateACResistanceFactor (V2 - Corrected)
%
    ========================================================================

% Description:
% Calculates the skin effect (ys or k_skin) and proximity effect (yp or
% k_proximity) factors based on the IEC 60287 standard. These factors
    are
% used to determine AC resistance from DC resistance.
```

```
% R_AC = R_DC * (1 + k_skin + k_proximity)
%
% Inputs:
%   freq      - System frequency [Hz]
%   diameter  - Conductor outer diameter [m]
%   sigma     - Electrical conductivity of the material [S*m/mm^2]
%
% Outputs:
%   k_skin      - Skin effect factor (dimensionless)
%   k_proximity - Proximity effect factor (dimensionless)
%
    ========================================================================


% --- Skin Effect Calculation (Based on IEC 60287-1-1) ---

% 1. Calculate DC resistance per meter [Ohm/m]
cross_section_m2 = pi * (diameter/2)^2; % [m^2]
sigma_Sm = sigma * 1e6; % Convert [S*m/mm^2] to [S/m]
R_dc_per_meter = 1 / (sigma_Sm * cross_section_m2);

% 2. Calculate the skin effect parameter x_s^2
if R_dc_per_meter == 0
    x_s_sq = 0;
else
    x_s_sq = (8 * pi * freq / R_dc_per_meter) * 1e-7;
end

% 3. Calculate the skin effect factor k_skin (also called ys)
k_skin = (x_s_sq^2) / (192 + 0.8 * x_s_sq^2);


% --- Proximity Effect Calculation (Based on IEC 60287-1-1) ---

% 1. The proximity effect parameter x_p^2 is identical to x_s^2
x_p_sq = x_s_sq;

% 2. The proximity factor depends on cable arrangement. The following is
    a
%    well-established approximation for three single-core cables in
   trefoil
%    (touching) formation, where center-to-center spacing 's' equals '
   diameter'.
%    For this arrangement, (diameter/spacing) = 1.
if x_p_sq == 0
    k_proximity = 0;
else
    k_proximity = (x_p_sq^2 / (192 + 0.8*x_p_sq^2)) * ...
                  ( 0.312 + (1.18 / ((x_p_sq^2 / (192 + 0.8*x_p_sq^2)) +
    0.27)) );
end

end
```

D.6 A4_ThermalEquilibrium.m

```
function T_eq = A4_ThermalEquilibrium(I, R_20, alpha, T_ref, envParams,
    lineLength, r_inner, r_outer)
%
    ========================================================================

% FUNCTION: A4_ThermalEquilibrium (V4 - Iterative)
%
    ========================================================================

% Description:
% Calculates the equilibrium temperature (T_eq) of a conductor for a
    given
% current. It iteratively solves the heat balance equation:
%   P_generated(T_eq) = P_dissipated(T_eq)
%
% MODIFIED:
% - The solving method is now iterative using fzero, which is more
    robust
%   for complex dissipation models (like radiation and convection).
%
    ========================================================================


% Define the heat balance equation as a function handle.
% The root of this function (where it equals zero) is the equilibrium
    temp.
% balance(T) = P_generated(T) - P_dissipated(T)
heat_balance_eq = @(T) (A3_ResistanceTemperatureCorrection(R_20, alpha,
    T, T_ref) * I^2) ...
                       - A7_HeatDissipation(T, envParams, r_outer,
    lineLength);

% Use MATLAB's built-in root finder 'fzero' to solve for T_eq.
% We provide an initial guess, e.g., the ambient temperature.
try
    options = optimset('Display', 'off'); % Suppress fzero output
    T_eq = fzero(heat_balance_eq, envParams.T_env, options);
catch
    % If fzero fails (e.g., no solution), return NaN
    T_eq = NaN;
    warning('Thermal equilibrium could not be found for I = %.1f A.', I)
    ;
end

end
```

D.7 A5_HeatingCurves.m

```
function A5_HeatingCurves(I_max, T_mat, envParams, R_20_DC, R_20_AC,
    alpha, T_ref, lineLength, r_inner, r_outer, materialName,
    insulationName)
%
    ========================================================================

% FUNCTION: A5_HeatingCurves (V5 - Corrected)
```

```
%
    ========================================================================

% Description:
% Generates plots showing the conductor's equilibrium temperature vs.
    current.
% MODIFIED: The input variable 'length' was renamed to 'lineLength' to
%           avoid conflict with MATLAB's built-in length() function,
    which
%           was causing a runtime error.
%
    ========================================================================


% 1. Create a vector of currents to plot
current_vector = linspace(0, I_max * 1.2, 100);

% 2. Calculate equilibrium temperatures for both DC and AC resistance
temp_vector_dc = zeros(size(current_vector));
temp_vector_ac = zeros(size(current_vector));

% The loop now correctly calls the built-in 'length' function
for i = 1:length(current_vector)
    % The variable 'lineLength' is correctly passed to the next function
    temp_vector_dc(i) = A4_ThermalEquilibrium(current_vector(i), R_20_DC
    , alpha, T_ref, envParams, lineLength, r_inner, r_outer);
    temp_vector_ac(i) = A4_ThermalEquilibrium(current_vector(i), R_20_AC
    , alpha, T_ref, envParams, lineLength, r_inner, r_outer);
end

% 3. Create the plot
figure;
hold on;

% Plot the curves
plot(current_vector, temp_vector_dc, 'b-', 'LineWidth', 2, 'DisplayName
    ', 'Equilibrium Temp (DC)');
plot(current_vector, temp_vector_ac, 'r-', 'LineWidth', 2, 'DisplayName
    ', 'Equilibrium Temp (AC)');

% Plot safety and reference lines
line([0, I_max * 1.2], [T_mat, T_mat], 'Color', 'k', 'LineStyle', '--',
    'LineWidth', 1.5, 'DisplayName', sprintf('Max Temp (%.0fC)', T_mat));
line([I_max, I_max], [envParams.T_env, T_mat], 'Color', [0.4660 0.6740
    0.1880], 'LineStyle', ':', 'LineWidth', 1.5, 'DisplayName', sprintf('
    Max AC Current (%.1f A)', I_max));

% Add marker at the intersection
plot(I_max, T_mat, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 8, '
    HandleVisibility', 'off');

% Formatting
title(sprintf('Steady-State Heating Curve (%s)', envParams.scenario));
subtitle(sprintf('%s Conductor, %s Insulation', materialName,
    insulationName));
xlabel('Current (A)');
ylabel('Conductor Temperature (C)');
legend('show', 'Location', 'southeast');
```

```
grid on;
box on;
ylim([envParams.T_env - 10, T_mat + 20]);
xlim([0, I_max * 1.25]);
hold off;

end
```

D.8 A6_TransientHeating.m

```
function [time, temp] = A6_TransientHeating(I, time_span, envParams, R_20, alpha, T_ref, length, r_inner, r_outer, mass, cp)
%
    ========================================================================

% FUNCTION: A6_TransientHeating (V5 - FIXED)
%
    ========================================================================

% Description:
% Models the temperature of a conductor over time.
% FIXED: The call to the heat dissipation function has been corrected from
% the erroneous 'A7_HeatGain' to the correct 'A7_HeatDissipation'.
%
% ODE: m*cp*(dT/dt) = P_generated(T) - P_dissipated(T)
%
    ========================================================================


    % Define the ODE as a nested function for the solver
    function dTdt = heat_balance_ode(~, T)
        % 1. Heat Generation at current temperature T
        R_T = A3_ResistanceTemperatureCorrection(R_20, alpha, T, T_ref);
        P_generated = R_T * I^2;

        % 2. Heat Dissipation at current temperature T
        % FIXED: Corrected function call from A7_HeatGain to A7_HeatDissipation
        P_dissipated = A7_HeatDissipation(T, envParams, r_outer, length);

        % 3. Rate of change of internal energy (dT/dt)
        dTdt = (P_generated - P_dissipated) / (mass * cp);
    end

% Use MATLAB's ODE solver (ode45)
options = odeset('RelTol', 1e-6, 'AbsTol', 1e-6);
% The initial temperature for the simulation is the ambient temperature
[time, temp] = ode45(@heat_balance_ode, time_span, envParams.T_env, options);

end
```

D.9 A7_HeatDissipation.m

```
function P_diss = A7_HeatDissipation(T_conductor, envParams, r_outer, length)
%
    ========================================================================

% FUNCTION: A7_HeatDissipation (Module)
%
    ========================================================================

% Description:
% This is the central module for calculating the net heat dissipation from
% the conductor surface to the environment. It calls the appropriate
% physical model based on the installation scenario defined in envParams.
%
% Inputs:
%   T_conductor - The temperature of the conductor surface [C]
%   envParams   - A struct containing all environment-specific parameters
%   r_outer     - The outer radius of the cable (including insulation) [m]
%   length      - The length of the conductor [m]
%
% Output:
%   P_diss      - The net heat dissipated from the cable [W].
%                 (Positive for net cooling, negative for net heating)
%
    ========================================================================


% The main function acts as a router to the correct sub-function
switch envParams.scenario
    case 'underground'
        P_diss = dissipation_underground(T_conductor, envParams.T_env, envParams.rho_soil, envParams.burial_depth, r_outer, length);
    case 'overhead'
        P_diss = dissipation_overhead(T_conductor, envParams.T_env, envParams.wind_speed, envParams.solar_irradiance, envParams.emissivity, envParams.absorptivity, r_outer, length);
    case 'building'
        % For a building, we model it as overhead but with no wind/sun and apply a grouping factor
        P_diss = dissipation_overhead(T_conductor, envParams.T_env, 0, 0, envParams.emissivity, envParams.absorptivity, r_outer, length);
        % Apply the derating factor to the dissipation capability
        if isfield(envParams, 'grouping_factor')
            P_diss = P_diss * envParams.grouping_factor;
        end
end
end

% --- SUB-FUNCTIONS for each scenario ---

function P_diss = dissipation_underground(T_conductor, T_soil, rho_soil, H, r_outer, L)
```

```
% Model based on IEC 60287 for buried cables (simplified)
% P = (T_conductor - T_ambient) / R_thermal
% R_thermal for a single buried cable is rho_soil/(2*pi) * log(2H/r)
    if H <= r_outer
        P_diss = 0; % Cable is not buried, avoid log error
        return;
    end
    thermal_resistance = (rho_soil / (2 * pi)) * log(2 * H / r_outer);
    P_diss = (L * (T_conductor - T_soil)) / thermal_resistance;
end

function P_diss = dissipation_overhead(T_conductor, T_air, V_wind,
   Q_solar, epsilon, alpha, r_outer, L)
% Model based on IEEE 738 standard for overhead lines
    D_outer = r_outer * 2;
    P_conv = convection_overhead(T_conductor, T_air, V_wind, D_outer, L)
   ;
    P_rad = radiation_overhead(T_conductor, T_air, epsilon, D_outer, L);
    P_solar = solar_gain(Q_solar, alpha, D_outer, L);

    % Net dissipation is cooling (convection + radiation) minus heating
   (solar)
    P_diss = (P_conv + P_rad) - P_solar;
end

function P_conv = convection_overhead(T_conductor, T_air, V_wind,
   D_outer, L)
% Simplified convection model
    k_air = 0.026; % Thermal conductivity of air [W/(m*K)]

    % Nusselt number correlations (simplified)
    if V_wind > 0
        % Forced convection
        Re = V_wind * D_outer / 1.5e-5; % Reynolds number, kinematic
   viscosity of air ~1.5e-5
        Nu = 0.3 + (0.62 * Re^0.5 * 0.71^0.33) / (1 + (0.4/0.71)^0.66)
   ^0.25; % Prandtl for air ~0.71
    else
        % Natural convection
        Gr = 9.81 * (1/(T_air+273.15)) * abs(T_conductor - T_air) *
   D_outer^3 / (1.5e-5)^2; % Grashof
        Nu = (0.6 + (0.387 * (Gr*0.71)^0.166) / (1 + (0.559/0.71)^0.562)
   ^0.296)^2;
    end

    h = (k_air / D_outer) * Nu; % Convective heat transfer coefficient
    surface_area = pi * D_outer * L;
    P_conv = h * surface_area * (T_conductor - T_air);
end

function P_rad = radiation_overhead(T_conductor, T_air, epsilon, D_outer
   , L)
% Radiation based on Stefan-Boltzmann law
    sigma_sb = 5.67e-8; % Stefan-Boltzmann constant [W/(m^2*K^4)]
    surface_area = pi * D_outer * L;

    % Temperatures must be in Kelvin
    T_cond_K = T_conductor + 273.15;
```

```
    T_air_K = T_air + 273.15;

    P_rad = sigma_sb * epsilon * surface_area * (T_cond_K^4 - T_air_K^4);
end

function P_solar = solar_gain(Q_solar, alpha, D_outer, L)
% Solar heat gain on the projected area of the conductor
    projected_area = D_outer * L;
    P_solar = Q_solar * alpha * projected_area;
end
```

D.10 B1_VoltageDropAnalysis.m

```
%
    ========================================================================

% MAIN SCRIPT for Voltage Drop Analysis (V3.1 - Mode Selection)
%
    ========================================================================

% MODIFIED:
% - Added a selection menu at the start to allow the user to choose
%   between a 'Full Installation' analysis (multi-segment) and a
%   'Single Line' analysis (the previous, simpler version).
% - The script now contains both workflows.
%
    ========================================================================


%% --- Cleanup ---
% clc; clear; close all; % Controlled by master script

%% --- User Selection of Analysis Mode ---
if ~exist('vd_analysisMode', 'var')
    vd_analysisMode = centeredMenu('Select Voltage Drop Analysis Type:',
    'Full Installation Analysis (Multi-Segment)', 'Simple Single Line
    Analysis');
end

if vd_analysisMode == 0
    disp('No analysis type selected. Exiting.');
    return;
end

%% --- Execute Selected Analysis ---
if vd_analysisMode == 1
    % --- RUN FULL INSTALLATION ANALYSIS (MULTI-SEGMENT) ---
    run_full_installation_analysis();
else
    % --- RUN SIMPLE SINGLE LINE ANALYSIS ---
    run_single_line_analysis();
end


```

```
%% ========================================================================
%                      WORKFLOW 1: FULL INSTALLATION
% ========================================================================
function run_full_installation_analysis()
    % --- Initialize Installation Structure ---
    installationSegments = {
        'LV Distribution Line',
        'Individual Connection Line',
        'Main Feeder Line (LGA)',
        'Individual Branch Line (DI)',
        'Internal Circuits'
    };
    numSegments = length(installationSegments);
    results = cell(numSegments, 1);

    % --- Define Physical Constants ---
    rho_copper = 0.0172; sigma_copper = 56;
    rho_aluminum = 0.0282; sigma_aluminum = 36;

    % --- Get Global Parameters ---
    disp('--- Entering Global Parameters for Full Installation ---');
    U_source_initial = input('Enter source voltage at LV Distribution Line [V] (e.g., 230 or 400): ');
    loadCurrent = input('Enter final load current [A] (e.g., 30): ');
    cos_phi = input('Enter final load power factor (e.g., 0.95): ');
    frequency = input('Enter AC frequency [Hz] (e.g., 50): ');

    lineTypeChoice = centeredMenu('Select Dominant System Type:', 'Single-Phase', 'Three-Phase');
    if lineTypeChoice == 1, lineType = 'single-phase'; else, lineType = 'three-phase'; end

    % --- Data Collection Loop for Each Segment ---
    voltage_at_start_of_segment = U_source_initial;
    for i = 1:numSegments
        fprintf('\n--- Input for Segment %d: %s ---\n', i, installationSegments{i});
        segmentData.name = installationSegments{i};
        segmentData.length = input(sprintf('Enter length [m] for %s: ', segmentData.name));
        segmentData.crossSection = input(sprintf('Enter cross-section [mm^2] for %s: ', segmentData.name));
        materialChoice = centeredMenu(['Select Material for ' segmentData.name], 'Copper', 'Aluminum');
        if materialChoice == 0, disp('Selection cancelled. Aborting.'); return; end
        switch materialChoice, case 1, segmentData.resistivity = rho_copper; segmentData.sigma = sigma_copper; case 2, segmentData.resistivity = rho_aluminum; segmentData.sigma = sigma_aluminum; end
        segmentData.reactance_per_meter = input(sprintf('Enter reactance [Ohm/m] for %s (e.g., 0.0001): ', segmentData.name));

        r_dc = segmentData.resistivity / segmentData.crossSection;
        diameter_m = sqrt(4 * segmentData.crossSection / pi) / 1000;
```

```
        [k_skin, k_prox] = A3b_AC_Resistance_Factor(frequency, diameter_m, segmentData.sigma);
        r_ac = r_dc * (1 + k_skin + k_prox);

        segmentData.voltageDrop = B2_calculateExactVoltageDrop(lineType, segmentData.length, loadCurrent, r_ac, segmentData.reactance_per_meter, cos_phi);
        segmentData.voltage_in = voltage_at_start_of_segment;
        segmentData.voltage_out = voltage_at_start_of_segment - segmentData.voltageDrop;
        voltage_at_start_of_segment = segmentData.voltage_out;
        results{i} = segmentData;
    end

    % --- Display Summary and Compliance ---
    display_summary_table(results, U_source_initial);
    B4b_checkInstallationCompliance(results, U_source_initial);
    B6_plotInstallationProfile(results, U_source_initial);
end

function display_summary_table(results, U_source_initial)
    fprintf('\n\n--- COMPLETE INSTALLATION VOLTAGE DROP SUMMARY ---\n');
    fprintf('================================================================================\n');
    fprintf('%-30s | %-10s | %-10s | %-12s | %-12s\n', 'Segment', 'Length (m)', 'Drop (V)', 'Voltage In (V)', 'Voltage Out (V)');
    fprintf('--------------------------------------------------------------------------------\n');
    cumulativeDrop = 0;
    for i = 1:length(results)
        data = results{i};
        cumulativeDrop = cumulativeDrop + data.voltageDrop;
        fprintf('%-30s | %-10.1f | %-10.2f | %-12.2f | %-12.2f\n', data.name, data.length, data.voltageDrop, data.voltage_in, data.voltage_out);
    end
    fprintf('================================================================================\n');
    fprintf('Total Voltage Drop from Source to Final Load: %.2f V (%.2f%%)\n', cumulativeDrop, (cumulativeDrop/U_source_initial)*100);
end


%% ========================================================================
%                     WORKFLOW 2: SINGLE LINE
% ========================================================================

function run_single_line_analysis()
    % --- Define Physical Constants ---
    rho_copper = 0.0172;   sigma_copper = 56;
    rho_aluminum = 0.0282; sigma_aluminum = 36;

```

```
    % --- User Input ---
    fprintf('\n--- Input for Single Line Analysis ---\n');
    lineTypeChoice = centeredMenu('Select Line Type:', 'Single-Phase', '
   Three-Phase');
    if lineTypeChoice == 1
        lineType = 'single-phase';
        U_source = input('Enter single-phase source voltage (L-N) [V] (e
   .g., 230): ');
    else
        lineType = 'three-phase';
        U_source = input('Enter three-phase source voltage (L-L) [V] (e.
   g., 400): ');
    end
    loadCurrent = input('Enter load current [A] (e.g., 25): ');
    cos_phi = input('Enter load power factor (e.g., 0.9): ');
    lineLength = input('Enter total line length [m] (e.g., 150): ');
    reactance = input('Enter line reactance per unit length [Ohm/m] (e.g
   ., 0.0001): ');
    frequency = input('Enter AC frequency [Hz] (e.g., 50): ');
    materialChoice = centeredMenu('Select Conductor Material:', 'Copper
   ', 'Aluminum');
    crossSection = input('Enter conductor cross-section [mm^2] (e.g.,
   16): ');
    circuitChoice = centeredMenu('Select Circuit Type (for REBT limits)
   :', 'Lighting', 'Power (Other uses)');
    if materialChoice == 0 || circuitChoice == 0, disp('Invalid
   selection. Aborting.'); return; end

    % --- Set Parameters ---
    switch materialChoice, case 1, resistivity = rho_copper; sigma =
   sigma_copper; case 2, resistivity = rho_aluminum; sigma =
   sigma_aluminum; end
    switch circuitChoice, case 1, circuitType = 'lighting'; case 2,
   circuitType = 'power'; end

    % --- Resistance Calculations (DC and AC) ---
    resistance_dc_per_meter = resistivity / crossSection;
    diameter_m = sqrt(4 * crossSection / pi) / 1000;
    [k_skin, k_proximity] = A3b_AC_Resistance_Factor(frequency,
   diameter_m, sigma);
    resistance_ac_per_meter = resistance_dc_per_meter * (1 + k_skin +
   k_proximity);

    % --- Perform Voltage Drop Calculations ---
    deltaU_exact_dc = B2_calculateExactVoltageDrop(lineType, lineLength,
    loadCurrent, resistance_dc_per_meter, reactance, cos_phi);
    deltaU_exact_ac = B2_calculateExactVoltageDrop(lineType, lineLength,
    loadCurrent, resistance_ac_per_meter, reactance, cos_phi);
    [isCompliant, percentDrop, limit] = B4_checkREBTCompliance(
   deltaU_exact_ac, U_source, circuitType);

    % --- Display Results ---
    fprintf('\n--- RESULTS for Single Line ---\n');
    fprintf('DC Resistance per meter: %.5f Ohm/m\n',
   resistance_dc_per_meter);
    fprintf('AC Resistance per meter: %.5f Ohm/m (Skin: %.2f%%, Prox:
   %.2f%%)\n', resistance_ac_per_meter, k_skin*100, k_proximity*100);
    fprintf('------------------------------------------------\n');
```

```
157     fprintf('Voltage Drop (using DC Resistance): %.2f V\n',
    deltaU_exact_dc);
158     fprintf('Voltage Drop (using AC Resistance): %.2f V (%.2f%%)\n',
    deltaU_exact_ac, percentDrop);
159     fprintf('Voltage at Load (based on AC drop): %.2f V\n', U_source -
    deltaU_exact_ac);
160     fprintf('REBT Compliance for %s: %s (Limit: %.1f%%)\n', circuitType,
     string(isCompliant), limit);
161     fprintf('------------------------------------------------\n');
162
163     % --- Plotting ---
164     B5_plotVoltageProfile(U_source, deltaU_exact_dc, deltaU_exact_ac,
    lineLength, circuitType);
165 end
```

D.11 C1_ConductorSizingAnalysis.m

```
1 %
    ========================================================================

2 % MAIN SCRIPT for Conductor Cross-Section Sizing (V2 - Integrated)
3 %
    ========================================================================

4 % MODIFIED:
5 % - Renamed from C1_CrossSection.m for consistency.
6 % - Integrated into the master project, controlled by '
    sizing_scenarioChoice'.
7 % - The script's workflow changes based on the selected scenario.
8 % - All user menus are now centered.
9 %
    ========================================================================


10
11 %% --- Cleanup and Initialization ---
12 % clc; clear; close all; % Controlled by master script
13
14 %% --- Scenario Selection ---
15 if ~exist('sizing_scenarioChoice', 'var')
16     sizing_scenarioChoice = centeredMenu('Select Sizing Criterion:', '
    Voltage Drop (Student A)', 'Current-Carrying Capacity (Student B)', '
    Economic Optimization (Student C)');
17 end
18
19 %% --- Define Physical & Economic Constants ---
20 sigma_copper = 56;
21 sigma_aluminum = 36;
22 LCC_years = 20;
23 hours_per_year = 4000;
24 cost_per_kWh = 0.15;
25
26 %% --- User Input ---
27 disp('--- Conductor Sizing Input ---');
28 lineTypeChoice = centeredMenu('Select Line Type:', 'Single-Phase', '
    Three-Phase');
```

```
if lineTypeChoice == 1, lineType = 'single-phase'; else, lineType = 'three-phase'; end
if strcmp(lineType, 'single-phase'), U_source = input('Enter source voltage (L-N) [V] (e.g., 230): ');
else, U_source = input('Enter source voltage (L-L) [V] (e.g., 400): '); end
loadCurrent = input('Enter load current [A] (e.g., 45): ');
cos_phi = input('Enter load power factor (e.g., 0.9): ');
lineLength = input('Enter total line length [m] (e.g., 200): ');
vd_limit_percent = input('Enter maximum allowable voltage drop [%] (e.g., 5): ');
materialChoice = centeredMenu('Select Conductor Material:', 'Copper', 'Aluminum');
if materialChoice == 1, material = 'Copper'; sigma = sigma_copper; else, material = 'Aluminum'; sigma = sigma_aluminum; end

deltaU_max = U_source * (vd_limit_percent / 100);

%% --- Execute Analysis Based on Scenario ---
switch sizing_scenarioChoice
    case 1 % Student A: Voltage Drop Criterion
        fprintf('\n--- ANALYSIS by VOLTAGE DROP Criterion ---\n');
        s_vd = C2_calculateRequiredSection(lineType, lineLength, loadCurrent, cos_phi, sigma, deltaU_max);
        fprintf('Theoretical section required for voltage drop: %.2f mm^2\n', s_vd);
        [s_final, ampacity] = C3_selectStandardSection(s_vd, material);
        fprintf('Selected standard section: %.2f mm^2\n', s_final);
        if loadCurrent > ampacity
            fprintf('WARNING: This section may not support the load current! (Load: %.1f A, Ampacity: %.1f A)\n', loadCurrent, ampacity);
        end
        final_vd = C6_verifyFinalDesign(lineType, lineLength, loadCurrent, cos_phi, sigma, s_final, U_source);
        fprintf('Final calculated voltage drop with %.2f mm^2 section: %.2f%%\n', s_final, final_vd);

    case 2 % Student B: Current-Carrying Capacity Criterion
        fprintf('\n--- ANALYSIS by CURRENT-CARRYING CAPACITY Criterion ---\n');
        [s_final, ~, all_sections] = C3_selectStandardSection(0, material); % Get the table

        s_ampacity_options = all_sections([all_sections.ampacity] >= loadCurrent, :);
        if isempty(s_ampacity_options)
            error('No standard conductor can support the specified load current of %.1f A.', loadCurrent);
        end
        s_final = s_ampacity_options(1).section; % Select the smallest section that meets ampacity

        fprintf('Minimum section required for ampacity (%.1f A): %.2f mm^2\n', loadCurrent, s_final);
        final_vd = C6_verifyFinalDesign(lineType, lineLength, loadCurrent, cos_phi, sigma, s_final, U_source);
        fprintf('Final calculated voltage drop with this section: %.2f %%\n', final_vd);
```

```
68          if final_vd > vd_limit_percent
69              fprintf('WARNING: This section does not meet the voltage
    drop requirement! (Calculated: %.2f%%, Limit: %.1f%%)\n', final_vd,
    vd_limit_percent);
70          end
71
72      case 3 % Student C: Economic Optimization
73          fprintf('\n--- ANALYSIS by ECONOMIC OPTIMIZATION Criterion ---\n
    ');
74          % 1. Technical Sizing First
75          s_vd = C2_calculateRequiredSection(lineType, lineLength,
    loadCurrent, cos_phi, sigma, deltaU_max);
76          [s_vd_std, ~] = C3_selectStandardSection(s_vd, material);
77
78          [~, ~, all_sections] = C3_selectStandardSection(0, material);
79          s_ampacity_options = all_sections([all_sections.ampacity] >=
    loadCurrent, :);
80          if isempty(s_ampacity_options), error('Load current %.1f A is
    too high for available conductors.', loadCurrent); end
81          s_amp_std = s_ampacity_options(1).section;
82
83          s_technical = max(s_vd_std, s_amp_std);
84          fprintf('Technically minimum required section: %.2f mm^2\n',
    s_technical);
85
86          % 2. Economic Analysis
87          idx_start = find([all_sections.section] == s_technical);
88          sections_to_analyze = all_sections(idx_start:end);
89
90          num_sections = numel(sections_to_analyze);
91          total_costs = zeros(1, num_sections); cable_costs = zeros(1,
    num_sections); loss_costs = zeros(1, num_sections);
92
93          for i = 1:num_sections
94              [total_costs(i), cable_costs(i), loss_costs(i)] =
    C4_calculateLifecycleCost(...
95                  sections_to_analyze(i), lineType, lineLength,
    loadCurrent, cos_phi, sigma,...
96                  LCC_years, hours_per_year, cost_per_kWh);
97          end
98
99          [min_cost, optimal_idx] = min(total_costs);
100         s_optimal = sections_to_analyze(optimal_idx).section;
101
102         fprintf('-------------------------------------------------\n');
103         fprintf('ECONOMIC OPTIMIZATION RESULTS:\n');
104         fprintf('Economically Optimal Section: %.2f mm^2\n', s_optimal);
105         fprintf('Associated Lifecycle Cost: %.2f\n', min_cost);
106         if s_optimal == s_technical
107             disp('The smallest technically compliant section is also the
     most economical.');
108         else
109             cost_at_technical = total_costs(1);
110             fprintf('Recommendation: Using a larger cable (%.2f mm^2 vs
    %.2f mm^2) could save %.2f over %d years.\n', s_optimal, s_technical,
     cost_at_technical-min_cost, LCC_years);
111         end
112
```

```
        C5_plotEconomicAnalysis(sections_to_analyze, cable_costs,
    loss_costs, total_costs, s_technical, s_optimal);
end

if sizing_scenarioChoice == 0, disp('No scenario selected. Exiting.');
    end
```

D.12 centeredMenu.m

```
function choice = centeredMenu(title, varargin)
%
    ============================================================================

% FUNCTION: centeredMenu (V2 - Robust UI Figure)
%
    ============================================================================

% Description:
% Creates a custom, modal dialog box that is always centered on the
    screen.
% This version uses uifigure for robust control over position and
    behavior,
% avoiding issues with the legacy 'menu' function.
%
% Inputs:
%   title    - The string for the title of the menu box.
%   varargin - A variable number of strings for the menu options.
%
% Output:
%   choice   - The index of the selected option (1, 2, 3...), or 0 if
%              the user closes the window.
%
    ============================================================================


% --- Define Figure and Button Geometry ---
numButtons = length(varargin);
figHeight = 50 + numButtons * 40 + 20; % Top padding + buttons + bottom
    padding
figWidth = 350;
buttonHeight = 30;
buttonWidth = figWidth - 60; % With padding on each side
buttonX = (figWidth - buttonWidth) / 2;

% --- Create a blank, invisible figure ---
fig = uifigure('Visible', 'off', ...
               'Name', title, ...
               'WindowStyle', 'modal', ... % Blocks interaction with
    other windows
               'Resize', 'off', ...
               'NumberTitle', 'off', ...
               'MenuBar', 'none', ...
               'ToolBar', 'none');

% --- Center the Figure on the Screen ---
screenSize = get(0, 'ScreenSize');
```

```
screenWidth = screenSize(3);
screenHeight = screenSize(4);
figX = (screenWidth - figWidth) / 2;
figY = (screenHeight - figHeight) / 2;
fig.Position = [figX, figY, figWidth, figHeight];

% --- Create Buttons for Each Option ---
for i = 1:numButtons
    % Position buttons from the top of the figure down
    buttonY = figHeight - 50 - ((i-1) * 40);

    uibutton(fig, 'Text', varargin{i}, ...
             'Position', [buttonX, buttonY, buttonWidth, buttonHeight], ...
             'ButtonPushedFcn', @(btn, event) buttonCallback(i));
end

% --- Store the choice in the figure's app data (safer than global) ---
fig.UserData.choice = 0; % Default to 0 (if window is closed)

% --- Define the Nested Callback Function ---
    function buttonCallback(buttonIndex)
        % When a button is pushed, store its index and resume the script
        fig.UserData.choice = buttonIndex;
        uiresume(fig);
    end

% --- Make the figure visible and wait for user input ---
fig.Visible = 'on';
uiwait(fig); % Pauses execution until uiresume is called or fig is closed

% --- Retrieve the choice and delete the figure to clean up ---
if isvalid(fig)
    choice = fig.UserData.choice;
    delete(fig);
else
    % This handles the case where the user closes the window with the 'X'
    choice = 0;
end

end
```